

AWS EC2 배포 응용



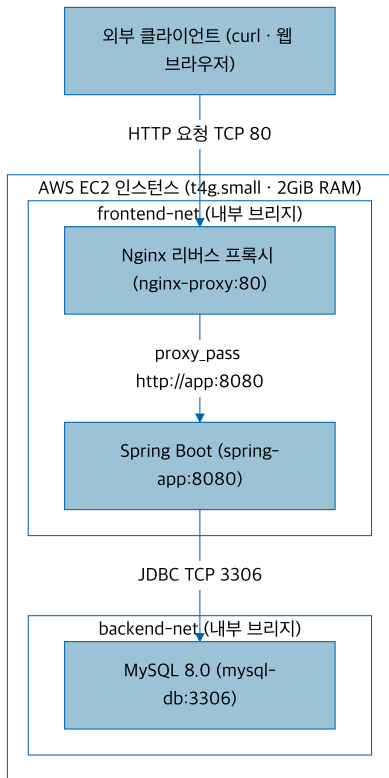
학습 목표

- `t4g.small` 인스턴스를 새로 프로비저닝하고 Docker 엔진을 설치해 3개 컨테이너 구동에 필요한 2GiB 메모리를 확보합니다
- Nginx 리버스 프록시 설정 파일을 작성해 80 포트 요청을 내부 스프링부트 컨테이너로 중계합니다
- `frontend-net` 과 `backend-net` 2계층 브리지로 웹, 애플리케이션, 데이터베이스를 격리합니다
- 서비스별 `deploy.resources.limits.memory` 상한을 배분해 인스턴스 전체의 OOM 발생을 차단합니다
- 80 포트 응답과 8080 직접 접속 차단을 대조 검증하고 불륨을 보존한 채 스택을 정리합니다

단일 인스턴스에 웹 서버, 애플리케이션, 데이터베이스를 함께 올리면 진입 경로 통제와 메모리 예산이 곧 안정성을 결정합니다. 본 차시는 3-Layer 네트워크 격리와 서비스별 메모리 상한을 하나의 `compose.yaml` 로 완성합니다.

핵심 개념 및 구조

단일 인스턴스 3-Layer 컨테이너 아키텍처



⇄ 단일 진입점

외부에 매핑된 호스트 포트는 Nginx의 **80** 하나뿐입니다

≡ 2계층 브리지

Nginx는 DB 네트워크에 소속되지 않아 직접 접근 경로가 없습니다

⇒ 포트 미노출

app과 **db**는 **ports**를 선언하지 않고 내부 DNS로만 통신합니다

3-Layer 계층별 역할과 네트워크 격리

계층 (현실 개념)	핵심 역할	포트 노출 정책	소속 네트워크
Web Tier (안내데스크)	외부 HTTP 요청 접수 및 백엔드 프록시 중계	호스트 80 포트 유일 개방	<code>frontend-net</code>
App Tier (사무실 직원)	비즈니스 로직 처리 및 데이터베이스 연동	포트 미노출 (내부 8080 수신)	<code>frontend-net</code> · <code>backend-net</code>
DB Tier (중앙 금고)	영속 데이터 저장 및 트랜잭션 관리	포트 미노출 (내부 3306 수신)	<code>backend-net</code>

2계층 브리지 격리로 Nginx는 DB에 직접 접근할 수 없으며, 두 네트워크를 연결하는 `app` 만이 DB와 통신할 수 있어 외부 공격 표면을 최소화합니다.

EC2 공인 IP: 동적 공인 IP vs 탄력적 IP (Elastic IP)

구분	동적 공인 IP (Public IP)	탄력적 IP (Elastic IP, EIP)
IP 고정 여부	인스턴스 중지 시 즉시 회수, 재시작 시 새 IP 할당	인스턴스 중지·재시작에도 유지되는 고정 공인 IP
주요 용도	단기 개발·테스트, IP 변경 무관 환경	운영 환경, 도메인 DNS (A 레코드) 연동 필수
과금 가드레일	실행 중인 인스턴스 1개 무료 할당	미연결 또는 중지된 인스턴스 방치 시 유휴 요금 발생
실습 적용	채택 (CLI로 재기동 시 <code>\$PUBLIC_IP</code> 동적 조회)	미채택 (실습 종료 후 유휴 과금 위험 방지)

과금 가드레일: 탄력적 IP는 AWS IPv4 자원 낭비 방지를 위해 실행 중인 인스턴스에 연결되지 않은 유휴 상태일 때 시간당 요금이 청구됩니다. 본 실습에서는 과금 사고를 방지하기 위해 동적 공인 IP와 CLI 조회를 채택합니다.

OOM(Out Of Memory)과 커널 OOM Killer 메커니즘

⚠ OOM과 OOM Killer

물리 RAM 고갈 시 커널 패닉을 방지하기 위해 `oom_score`가 가장 높은 프로세스를 `SIGKILL(9)`로 강제 사살하는 OS 최후의 방어선입니다.

📖 Exit Code 137 판별

시그널 종료 코드 공식 $128 + \text{Signal}(9) = 137$ 에 따라 컨테이너의 `Exit 137`은 OOM Killer에 의한 강제 사살을 의미합니다.

구분	상한 미지정 (호스트 OOM)	상한 지정 (컨테이너 OOM)
사살 대상	<code>sshd</code> · <code>dockerd</code> 사살 (서버 마비)	해당 cgroup 격리 종료 (<code>OOMKilled</code>)
영향 범위	SSH 세션 단절 및 인스턴스 마비	해당 컨테이너 단독 중단
보호 효과	시스템 전체 장애로 번짐	타 서비스와 호스트 OS 보호

메모리 상한은 프로세스를 불사신으로 만드는 것이 아니라, **하나가 죽더라도 호스트와 다른 서비스로 장애가 번지지 않게 격리**하는 방화벽입니다.

단일 인스턴스 리소스 제약과 OOM 방지 개요

🔗 t4g.small 메모리 제약 (2GiB)

Nginx, Spring Boot, MySQL 3개 컨테이너와 OS 커널이 2GiB RAM을 공유합니다

⚠️ OOM Killer 강제 종료 위험

상한 미지정 시 MySQL·JVM이 메모리를 과다 점유해 커널이 프로세스를 사살합니다

📊 deploy.resources.limits 강제

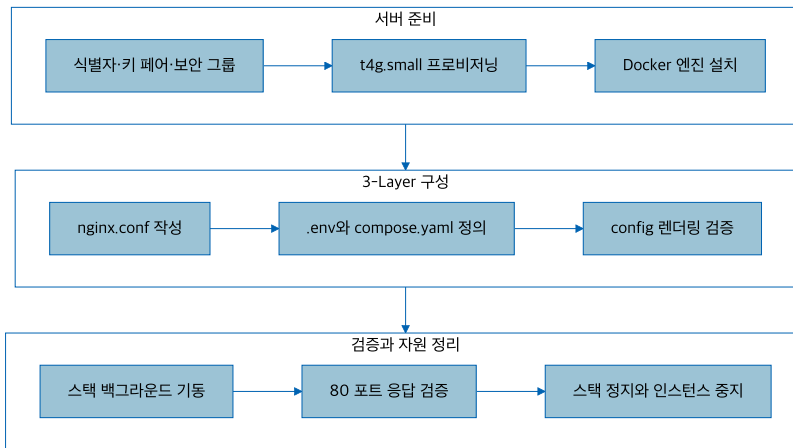
서비스별 메모리 상한(Nginx 64M, App 896M, DB 512M)으로 폭주를 방지합니다

서비스	메모리 상한	역할
nginx	64M	경량 프록시 중계 전용
app	896M	Spring Boot (힙 672MB)
db	512M	MySQL 버퍼 풀 축소
OS · 데몬	576M	커널 및 Docker 버퍼

서비스별 세부 메모리 튜닝 옵션(InnoDB 버퍼 풀, `MaxRAMPercentage`)과 cgroup 동작 원리는 Appendix의 메모리 예산 상세 설계에서 다룹니다.

실습 단계

실습 전체 흐름: 인스턴스 프로비저닝부터 자원 정리까지



구간	검증 지점
서버 준비	<code>uname -m</code> 이 <code>aarch64</code> , <code>free -h</code> 가 약 2GiB, Compose 플러그인 응답
구성과 검증	80 포트 <code>200 OK</code> 응답, 8080 직접 접속은 타임아웃

실습 식별자 정의와 키 페어 발급

```
export STUDENT_ID="student01"
export AWS_PROFILE="$STUDENT_ID"
export AWS_REGION="ap-northeast-2"
export AWS_PAGER=""

export MY_KEY_NAME="${STUDENT_ID}-key"
export MY_SG_NAME="${STUDENT_ID}-web-sg"
export MY_INSTANCE_NAME="${STUDENT_ID}-compose-ec2"

mkdir -p ~/workspace/aws-lab && cd ~/workspace/aws-lab

if [ ! -f "$MY_KEY_NAME".pem ]; then
  aws ec2 create-key-pair --key-name "$MY_KEY_NAME" \
    --query "KeyMaterial" --output text > "$MY_KEY_NAME".pem
  chmod 400 "$MY_KEY_NAME".pem
fi
```

변수	파생 규칙
MY_KEY_NAME	\${STUDENT_ID}-key
MY_SG_NAME	\${STUDENT_ID}-web-sg
MY_INSTANCE_NAME	\${STUDENT_ID}-compose-ec2

공용 교육 계정이므로 배부받은 사용자명 하나에서 모든 자원 이름을 파생시키고 **Owner** 태그를 함께 부여합니다. 본인 값으로 바꾸는 곳은 **STUDENT_ID** 한 줄뿐입니다.

보안 그룹 준비와 인바운드 규칙 등록

```
export MY_SG_ID=$(aws ec2 describe-security-groups \
  --filters "Name=group-name,Values=$MY_SG_NAME" \
  --query "SecurityGroups[0].GroupId" --output text)

export MY_IP=$(curl -fsS https://checkip.amazonaws.com)

aws ec2 authorize-security-group-ingress --group-id "$MY_SG_ID" \
  --protocol tcp --port 22 --cidr "$MY_IP/32"
aws ec2 authorize-security-group-ingress --group-id "$MY_SG_ID" \
  --protocol tcp --port 80 --cidr 0.0.0.0/0
```

포트	허용 대역
22	내_공인_IP/32
80	0.0.0.0/0
8080	0.0.0.0/0

SSH(22)는 무차별 대입 공격을 막기 위해 현재 공인 IP **/32**로만 엽니다. 8080은 애플리케이션을 노출하려는 것이 아니라, 포트 매핑이 없으면 보안 그룹이 열려 있어도 접속되지 않음을 뒤에서 대조하기 위해 등록합니다.

t4g.small 인스턴스 프로비저닝과 공인 IP 확인

```
export AMI_ID=$(aws ssm get-parameter --name \
  /aws/service/canonical/ubuntu/server/26.04/stable/current/arm64/hvm/ebs-gp3/ami-id \
  --query "Parameter.Value" --output text)

export INSTANCE_ID=$(aws ec2 run-instances \
  --image-id "$AMI_ID" --instance-type t4g.small \
  --key-name "$MY_KEY_NAME" --security-group-ids "$MY_SG_ID" \
  --tag-specifications "ResourceType=instance,Tags=[{Key=Name,Value=$MY_INSTANCE_NAME},{Key=Owner,Value=$STUDENT_ID}]" \
  --query "Instances[0].InstanceId" --output text)

aws ec2 wait instance-running --instance-ids "$INSTANCE_ID"
aws ec2 wait instance-status-ok --instance-ids "$INSTANCE_ID"

export PUBLIC_IP=$(aws ec2 describe-instances --instance-ids "$INSTANCE_ID" \
  --query "Reservations[0].Instances[0].PublicIpAddress" --output text)
```

공식 waiter 사용

임의의 `sleep` 대신 `wait instance-status-ok`로 상태 검사 통과까지 대기합니다. 부팅과 상태 검사에 2~3분이 걸립니다.

동적 공인 IP

탄력적 IP를 연결하지 않았으므로 인스턴스를 중지·재시작할 때마다 공인 IP가 갱신됩니다. `$PUBLIC_IP`를 매번 다시 조회해 변수에 담습니다.

Docker 엔진 설치

```
ssh -i ./"$MY_KEY_NAME".pem \  
-o StrictHostKeyChecking=accept-new ubuntu@"$PUBLIC_IP"
```

— 원격 EC2 터미널 내부 실행 —

```
uname -m
```

```
free -h | head -2
```

```
curl -fsSL https://get.docker.com -o get-docker.sh
```

```
sudo sh get-docker.sh
```

검증 항목

기대 출력

uname -m

aarch64

free -h

Mem: 1.8Gi

공식 설치 스크립트 하나로 Docker 엔진과 Compose 플러그인이 함께 설치됩니다. ARM 기반 인스턴스이므로 이후 내려받는 이미지도 **arm64** 아키텍처여야 합니다.

원격 Docker 명령의 실행 권한

— 원격 EC2 터미널 내부 실행 —

```
sudo systemctl is-active docker
```

```
sudo docker --version
```

```
sudo docker compose version
```

```
docker ps      # sudo 없이 실행하면 거부된다
```

명령	결과
<code>systemctl is-active docker</code>	<code>active</code>
<code>docker ps</code> (sudo 생략)	<code>permission denied</code>

설치 직후 `ubuntu` 계정은 `docker` 그룹에 속하지 않아 데몬 소켓 접근이 거부됩니다. 이후 원격의 모든 Docker 명령은 `sudo docker` · `sudo docker compose`로 실행합니다.

`sudo usermod -aG docker ubuntu` 후 재접속하면 `sudo`를 생략할 수 있으나, `docker` 그룹 가입은 사실상 root 동등 권한이므로 선택 절차로만 다룹니다.

Nginx 리버스 프록시 설정: nginx.conf

```
events {  
    worker_connections 1024;  
}  
  
http {  
    server {  
        listen 80;  
  
        location / {  
            proxy_pass http://app:8080;  
            proxy_set_header Host $host;  
            proxy_set_header X-Real-IP $remote_addr;  
            proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;  
        }  
    }  
}
```

지시어	역할
<code>listen 80</code>	외부 표준 HTTP 포트 수신
<code>proxy_pass</code>	Compose 내부 DNS로 <code>app</code> 컨테이너 중계
<code>X-Real-IP</code>	원본 클라이언트 IP를 단일 값으로 전달
<code>X-Forwarded-For</code>	경유한 프록시 목록을 누적해 전달

프록시를 거치면 백엔드가 관측하는 출발지 IP가 Nginx 컨테이너 주소로 바뀌므로, 원본 공인 IP는 `X-Forwarded-For` 헤더로만 남습니다.

공통 환경변수 파일과 HikariCP 지연 연결 옵션

```
cat <<'EOF' > .env
DB_ROOT_PASSWORD=rootpass
DB_NAME=appdb
DB_USER=root
DB_PASSWORD=rootpass

APP_MESSAGE=Live on AWS EC2 via Docker Compose!

SPRING_DATASOURCE_HIKARI_INITIALIZATION_FAIL_TIMEOUT=-1
SPRING_DATASOURCE_HIKARI_CONNECTION_TIMEOUT=30000
EOF
cat .env
```

환경변수

역할

`INITIALIZATION_FAIL_TIMEOUT=-1`

DB 미기동 시에도
즉시 종료하지 않고
연결 재시도

`CONNECTION_TIMEOUT=30000`

커넥션 획득까지 최대
30초 대기

MySQL은 최초 기동 시 데이터 디렉터리를 초기화하므로
연결 수락까지 수십 초가 걸립니다. 기본 설정의
스프링부트는 이 구간에서 커넥션 풀 초기화에 실패해 `Exit`
`1`로 종료됩니다.

compose.yaml 서비스 정의: Nginx와 Spring Boot

```
services:
  nginx:
    image: nginx:alpine
    container_name: nginx-proxy
    restart: always
    ports: ["80:80"]
    volumes: [".:/nginx.conf:/etc/nginx/nginx.conf:ro"]
    depends_on: [app]
    deploy: { resources: { limits: { memory: 64M } } }
    networks: [frontend-net]

  app:
    image: ghcr.io/${GITHUB_USERNAME}/simple-back:latest
    container_name: spring-app
    restart: on-failure
    env_file: [.env]
    environment:
      PORT: 8080
      JAVA_TOOL_OPTIONS: "-XX:MaxRAMPercentage=75.0"
      SPRING_DATASOURCE_URL: "jdbc:mysql://db:3306/${DB_NAME}?useSSL=false"
      SPRING_DATASOURCE_USERNAME: ${DB_USER}
      SPRING_DATASOURCE_PASSWORD: ${DB_PASSWORD}
    depends_on: [db]
    deploy: { resources: { limits: { memory: 896M } } }
    networks: [frontend-net, backend-net]
```

설정 항목	역할
<code>nginx.ports</code>	인스턴스 외부로 개방되는 유일한 포트 매핑
<code>app</code> 포트 생략	외부 직접 접근을 차단하고 내부 통신만 허용

`app`은 `frontend-net`과 `backend-net`에 동시에 소속되어 두 계층을 잇는 유일한 서비스가 됩니다. JDBC URL에는 꼭 제약으로 생략한 `&allowPublicKeyRetrieval=true`를 함께 붙여야 MySQL 8 인증이 통과합니다.

MySQL 서비스 정의와 네트워크 격리

```
db:
  image: mysql:8.0
  container_name: mysql-db
  restart: always
  command: --innodb-buffer-pool-size=256M
  environment:
    MYSQL_ROOT_PASSWORD: ${DB_ROOT_PASSWORD}
    MYSQL_DATABASE: ${DB_NAME}
  volumes: ["db-data:/var/lib/mysql"]
  deploy:
    resources:
      limits: { memory: 512M }
      networks: [backend-net]
  volumes: { db-data: }
  networks:
    frontend-net:
    backend-net:
```

설정 항목	역할
<code>--innodb-buffer-pool-size=256M</code>	호스트 메모리에 비례하는 기본 버퍼 풀을 축소해 DB 메모리 잠식 방지
<code>networks:</code> <code>[backend-net]</code>	후단 네트워크에만 연결해 Nginx의 직접 접근 경로 제거

`db` 서비스에는 `ports` 를 선언하지 않았으므로 3306 포트가 호스트에 노출되지 않고 `backend-net` 내부에서만 도달 가능합니다.

사전 렌더링 검증과 전체 스택 구동

— 원격 EC2 터미널 내부 실행 —

```
sudo docker compose config
```

```
sudo docker compose up -d
```

```
sudo docker compose logs db | grep "ready for connections"
```

초기화 판별 기준

MySQL이 소켓과 TCP 연결을 수락할 준비를 마치면 `ready for connections` 로그가 출력됩니다

명령

설명

```
docker compose  
config
```

환경변수 치환 결과와 문법 오류를
기동 전에 렌더링해 검증

```
docker compose  
up -d
```

Nginx, App, DB 컨테이너를
백그라운드로 일괄 기동

```
docker compose  
logs db
```

초기화 로그에서 연결 수락 준비
시점을 직접 확인

렌더링 출력에서 `${DB_NAME}` 자리가 `appdb`로
치환되었는지 확인합니다. 치환에 실패하면 빈 문자열이
들어가 JDBC 접속 URL이 무효가 됩니다.

스프링부트 지연 연결 복구와 컨테이너 상태 점검

— 원격 EC2 터미널 내부 실행 —

```
sudo docker compose ps
```

```
sudo docker compose restart app
```

```
sudo docker compose ps
```

```
exit
```

`depends_on`은 컨테이너 기동 순서만 보장하고 애플리케이션의 준비 완료는 보장하지 않습니다.

컨테이너	기대 상태	확인 내용
nginx-proxy	Up	80 포트 수신 중
spring-app	Up	DB 연결 후 정상 가동
mysql-db	Up	3306 포트 내부 수신 중

MySQL 초기화가 완료되기 전에 `app` 컨테이너가 먼저
기동하면 커넥션 획득에 실패해 `Exit 1`로 종료될 수
있습니다. `restart app`으로 재기동합니다.

리버스 프록시 응답 검증과 백엔드 포트 격리 대조

```
# 로컬 호스트 터미널에서 실행
curl -i http://"$PUBLIC_IP"/
curl -i http://"$PUBLIC_IP"/users
```

```
# 8080 직접 접근 시도
curl -i --max-time 3 http://"$PUBLIC_IP":8080/ \
  || echo "8080 포트 직접 접속 차단 확인"
```

🌀 80 포트 정상 응답

클라이언트에서 Nginx, 스프링부트, MySQL로 이어지는 3-Layer 경로가 **200 OK**로 검증됩니다

✅ 8080 직접 접근 차단

app 서비스에 포트 매핑을 선언하지 않아 외부 우회 접근이 원천 차단됩니다

두 요청을 대조해야 프록시 경유 경로만 열려 있고 백엔드 포트는 닫혀 있다는 격리 상태가 입증됩니다.

컨테이너별 자원 점유 점검: docker stats

```
# — 원격 EC2 터미널 내부 실행 —  
sudo docker stats --no-stream
```

서비스	설정 상한	실제 사용량 관찰 기준
nginx-proxy	64MiB	5~15MiB 수준의 극저용량 유지
spring-app	896MiB	300~500MiB 수준에서 안정적 유지
mysql-db	512MiB	200~350MiB 수준 유지

세 컨테이너의 상한 합계는 1,472MB이며 나머지 576MB가 커널, OpenSSH, Docker 데몬 몫으로 남습니다. 실사용량이 상한에 근접하면 버퍼 풀이나 힙 비율을 재조정합니다.

프로세스 매핑과 스택 정지·재개

— 원격 EC2 터미널 내부 실행 —

```
sudo docker compose top
```

```
sudo docker compose stop
```

```
sudo docker compose ps
```

```
sudo docker compose start
```

```
sudo docker compose ps
```

```
exit
```

명령	설명
<code>docker compose top</code>	컨테이너 내부 프로세스가 호스트에서 어떤 PID로 격리 실행되는지 확인
<code>docker compose stop / start</code>	컨테이너와 볼륨을 유지한 채 프로세스만 정지하고 복구

`stop`은 컨테이너를 삭제하지 않고 프로세스만 종료하므로

`ps` 목록에 `Exited` 상태로 남으며, `start`로 동일한

컨테이너가 그대로 재기동됩니다.

볼륨 보존 종료와 컴퓨팅 과금 동결

로컬 호스트 터미널에서 실행

```
ssh -i ./"$MY_KEY_NAME".pem ubuntu@"$PUBLIC_IP" \  
"cd ~ && sudo docker compose down && sudo docker volume ls"
```

```
aws ec2 stop-instances --instance-ids "$INSTANCE_ID"
```

```
aws ec2 wait instance-stopped --instance-ids "$INSTANCE_ID"
```

데이터 연속성

명명된 볼륨 `db-data`를 유지하면 다음 실습에서 MySQL 데이터를 그대로 재사용할 수 있습니다

명령	볼륨 보존 여부	설명
<code>docker compose down</code>	보존됨	컨테이너와 네트워크만 정리
<code>docker compose down -v</code>	삭제됨	명명된 볼륨까지 영구 삭제

`-v` 플래그는 디스크에서 볼륨을 영구 삭제하므로 데이터 초기화가 필요한 상황이 아니면 운영 환경에서 사용하지 않습니다.

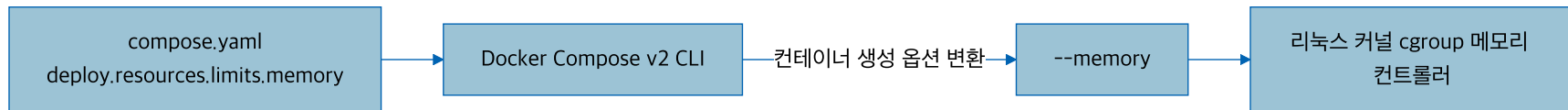
Appendix

단일 인스턴스 메모리 예산 상세 설계

서비스	메모리 상한	튜닝 옵션과 목적
nginx	64M	이벤트 루프 기반 경량 프록시, 중계 전용 최소 할당
app	896M	<code>-XX:MaxRAMPercentage=75.0</code> 으로 약 672MB를 힙에 배정
db	512M	<code>--innodb-buffer-pool-size=256M</code> 으로 버퍼 풀 축소
OS · 데몬 여유	576M	커널, OpenSSH, Docker 데몬 버퍼 확보
합계	2,048M	<code>t4g.small</code> 전체 메모리 2.0GiB

상한을 명시하지 않으면 MySQL이 호스트 메모리에 비례해 버퍼 풀을 잡고 JVM도 가용 메모리를 넓게 인식합니다.
총합이 물리 메모리를 넘는 순간 커널 OOM Killer가 프로세스를 강제 종료합니다.

deploy 리밋 동작 원리와 컨테이너 인지형 JVM



☞ 스펙의 출신

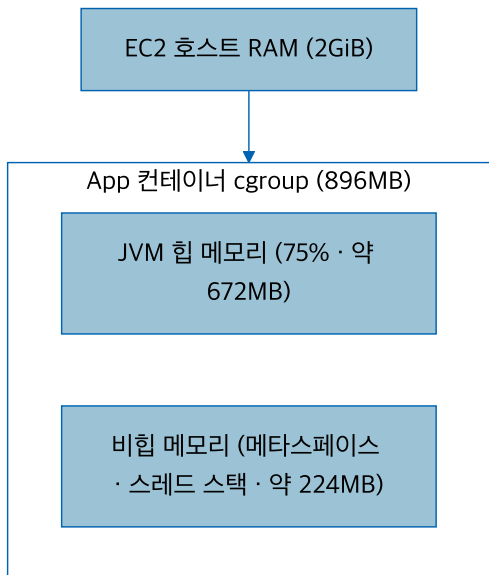
`deploy` 키는 원래 Docker Swarm의 `docker stack`
`deploy` 전용 스펙이었습니다

☒ 단일 호스트 적용

Compose v2 CLI가 `docker compose up` 시점에 직접
변환하므로 상한이 강제 적용됩니다

키	적용 범위
<code>deploy.resources.limits.memory</code>	Compose Specification 공식 표준, 단일 호스트와 Swarm 양쪽 동일 문법
<code>mem_limit</code>	Compose v1 계열 레거시 키, 단일 호스트 전용이며 Swarm에서는 무시

deploy 리밋 동작 원리와 컨테이너 인지형 JVM



기본값의 한계

JDK 10이후 JVM은 cgroup 상한을 자동 인식하지만 기본 `MaxRAMPercentage=25.0`은 약 224MB만 힙으로 배정합니다

힙 비율 상향

`JAVA_TOOL_OPTIONS`로 `-XX:MaxRAMPercentage=75.0`을 주입해 안정성과 처리량을 동시에 확보합니다

비힙 메모리 팽창으로 cgroup 896MB 상한을 넘어서면 JVM 예외 처리 없이 커널 OOM Killer가 `SIGKILL(9)`로 프로세스를 강제 사살(`Exit 137`)합니다.

트래픽 제어 기술 계층별 비교

구분	포트포워딩	리버스 프록시	로드밸런싱
동작 계층	L4 전송 계층, NAT 변환	L7 애플리케이션 계층, HTTP	L4 NLB 또는 L7 ALB
핵심 역할	호스트와 컨테이너 포트 매핑	백엔드 은닉, 헤더 보존, SSL 종단	트래픽 분산과고가용성 확보
본 실습 적용	Compose <code>ports: ["80:80"]</code>	Nginx <code>proxy_pass http://app:8080;</code>	단일 인스턴스에서는 미적용

포트포워딩은 패킷의 목적지 주소만 바꾸므로 HTTP 헤더를 읽지 못합니다. 클라이언트 원본 IP 보존과 백엔드 은닉은 `L7`에서 동작하는 리버스 프록시의 역할입니다.

트래픽 제어 기술 계층별 비교

🔒 보안 격리

백엔드 포트 8080과 내부 주소를 외부에 드러내지 않아 공격 표면을 축소합니다

</> L7 헤더 보존

클라이언트의 원본 공인 IP를 `X-Forwarded-For` 헤더로 전달해 감사와 보안 정책 수립을 지원합니다

🔗 SSL 종단과 캐싱

암호화 해독과 정적 자원 캐싱을 Nginx 단에서 처리해 백엔드 부하를 분리합니다

관점	프록시 없이 포트를 직접 개방	Nginx를 전면에 둔 구성
외부 노출 표면	애플리케이션 포트와 버전 정보가 그대로 노출	80 포트 하나만 개방되고 백엔드는 은닉
클라이언트 IP	백엔드가 관측하는 주소가 NAT 주소로 치환	<code>X-Forwarded-For</code> 헤더로 원본 IP 보존

포트포워딩은 L4 계층에서 주소와 포트를 단순 치환할 뿐이며, HTTP 헤더를 해석하지 못하므로 원본 IP 보존이나 경로 기반 분기가 불가능합니다.

Nginx의 HTTPS 처리: Let's Encrypt · Certbot 기반 SSL/TLS 종단

🏠 SSL 종단 (SSL Termination)

클라이언트-Nginx 구간만 HTTPS로 암호화하고, 내부 중계 (`proxy_pass`)는 사설망 평문 HTTP를 사용해 백엔드 부담을 제거합니다

🤖 Certbot 자동화 에이전트

인증서 발급, Nginx의 `listen 443 ssl;` 설정 삽입, 90일 주기 인증서 자동 갱신(cron/timer)을 자동 대행합니다

구성 요소	역할	동작 메커니즘
<code>ISRG Let's Encrypt</code>	무료 공인 인증 기관 (CA)	<code>ACME</code> 프로토콜 기반 자동 검증 후 90일 유효 <code>X.509</code> 인증서 발급
<code>Certbot</code>	공식 ACME 클라이언트	도메인 소유권 검증 후 웹서버 SSL 설정 자동 구성 및 갱신

Nginx의 HTTPS 처리: Let's Encrypt · Certbot 기반 SSL/TLS

종단

검증 단계	동작 메커니즘	세부 내용
1. 인증서 요청	ACME 클라이언트 (<code>Certbot</code>)	도메인(FQDN)에 대한 인증서 발급 요청 전달
2. <code>HTTP-01 challenge</code>	Let's Encrypt CA 검증	<code>http://<도메인>/.well-known/acme-challenge/<토큰></code> 조회
3. 소유권 확인	DNS A 레코드 대상 확인	도메인이 실제 해당 서버 IP를 가리키는지 확인 후 발급

공인 IP 단독 발급 불가: `HTTP-01 challenge`는 도메인 이름(FQDN) 소유권 검증을 전제하므로, 도메인이 연결되지 않은 순수 공인 IP(`http://$PUBLIC_IP`)로는 인증서를 발급할 수 없습니다. 본 실습이 80번 포트(HTTP)로 구성되는 이유입니다.

서브도메인 기반 다중 서비스 라우팅과 Nginx server_name

구분	동작 메커니즘
가상 호스트 라우팅	동일 IP와 포트로 유입된 요청을 <code>Host</code> 헤더 기준으로 <code>server_name</code> 블록에 분기
<code>SAN</code> 인증서	<code>-d api.example.com -d admin.example.com</code> 처럼 여러 도메인을 하나의 인증서에 결합
와일드카드 인증서	<code>*.example.com</code> 형태로 발급해 서브도메인이 유동적으로 늘어나는 구성에 대응

와일드카드 인증서는 `HTTP-01 challenge` 로 발급할 수 없고 DNS `TXT` 레코드를 등록하는 `DNS-01 challenge` 가 필요합니다. DNS 제공자 API 연동이 전제되므로 운영 복잡도가 함께 상승합니다.

핵심 기술 면접 질문

표준 필드와 레거시 키

`deploy.resources.limits.memory`와 레거시 `mem_limit`의 차이는 무엇인가요?

OOM Killer와 종료 코드

컨테이너 메모리 상한 초과 시 OOM Killer 동작 원리와 `Exit 137`의 의미는 무엇인가요?

힙 비율 과다 설정의 위험

`MaxRAMPercentage`를 95%이상으로 높게 설정하면 어떤 위험이 있나요?

Compose 볼륨 보존 여부

`docker compose down`과 `docker compose down -v`의 결정적 차이는 무엇인가요?

핵심 기술 면접 질문

공인 IP와 인증서 발급

EC2 공인 IP만으로 **Let's Encrypt** 인증서를 발급받을 수 없는 이유는 무엇인가요?

다중 서브도메인 HTTPS

서브도메인이 여러 개인 서비스에서 HTTPS를 적용하려면 어떻게 구성해야 하나요?

모범 답변

표준 필드와 레거시 키의 차이

`mem_limit`은 Compose v1 및 초기 v2에서 쓰이던 레거시 키로 단일 호스트 전용이며 Swarm 모드에서는 무시됩니다. `deploy.resources.limits.memory`는 최신 Compose Specification 공식 표준 필드로, 단일 호스트와 Swarm 클러스터 배포 양쪽에서 동일한 문법으로 cgroup 메모리 하드 리밋을 적용할 수 있어 실무 표준으로 사용됩니다.

OOM Killer 동작 원리와 Exit 137

컨테이너 프로세스가 cgroup 메모리 상한을 초과하면 커널은 시스템 다운을 방지하기 위해 해당 cgroup 내 `oom_score`가 가장 높은 프로세스에 `SIGKILL(9)`을 전송해 즉각 강제 사살합니다. 프로세스는 `128 + Signal(9) = 137` 공식에 따라 강제 종료되어 `Exit 137` 및 `OOMKilled`로 기록됩니다. 애플리케이션 레벨에서 힙 고갈 시 스택 트레이스를 남기는 JVM의 `java.lang.OutOfMemoryError`와 달리, 커널 OOM Killer는 프로세스를 가차 없이 즉각 강제 종료한다는 결정적 차이가 있습니다.

힙 비율을 과도하게 높일 때의 위험

JVM은 힙 외에도 클래스 메타데이터를 저장하는 메타스페이스, 스레드당 생성되는 스택 메모리, JIT 컴파일러 코드 캐시, 네이티브 버퍼 등 상당한 크기의 비힙 메모리를 소비합니다. 힙 비율을 95%로 설정하면 비힙 메모리가 cgroup 상한을 초과해, JVM 내부에서 `java.lang.OutOfMemoryError`가 발생하기도 전에 커널 OOM Killer가 `SIGKILL(9)`로 프로세스를 강제 사살(`Exit 137`)합니다.

모범 답변

볼륨 보존 여부의 차이

`docker compose down`은 컨테이너와 사용자 정의 네트워크만 삭제하고 데이터가 저장된 볼륨은 유지하므로 이후 다시 기동해도 DB 데이터가 복원됩니다. `-v` 플래그를 붙이면 명명된 볼륨까지 디스크에서 영구 삭제되므로 데이터 초기화가 반드시 필요한 상황이 아니라면 운영 환경에서 사용해서는 안 됩니다.

공인 IP로 인증서를 발급받을 수 없는 이유

`Let's Encrypt`는 `ACME` 프로토콜의 `HTTP-01 challenge`로 도메인 소유권을 검증하며, 이는 요청받은 도메인 이름의 DNS 레코드가 실제로 해당 서버를 가리키는지 확인하는 절차입니다. 공인 IP 자체는 소유권을 증명할 도메인 개념이 없어 검증 대상이 성립하지 않으므로 발급이 거부됩니다. HTTPS를 적용하려면 도메인을 구매해 해당 IP로 A 레코드를 연결한 뒤 인증서를 발급받아야 합니다.

다중 서브도메인 HTTPS 구성

Nginx 측에서는 `server_name` 지시어로 구분된 여러 `server` 블록을 두어 `Host` 헤더 기준으로 서브도메인별 백엔드에 `proxy_pass` 합니다. 인증서 측에서는 서브도메인 목록을 `-d` 옵션으로 나열해 하나의 SAN 인증서에 묶거나, 서브도메인이 유동적으로 늘어나는 경우 `*.example.com` 와일드카드 인증서를 `DNS-01 challenge` 방식으로 발급받아 대응합니다.

깊이 알아보기

🌀 Nginx proxy_pass 공식 명세

https://nginx.org/en/docs/http/nginx_http_proxy_module.html

⚙️ Docker Compose deploy 명세

<https://docs.docker.com/reference/compose-file/deploy/>

☁️ AWS EC2 탄력적 IP 설명서

<https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/elastic-ip-addresses-eip.html>

📖 Linux 커널 cgroup v2 메모리

<https://docs.kernel.org/admin-guide/cgroup-v2.html>

깊이 알아보기

📖 Let's Encrypt 공식 문서

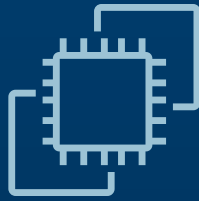
<https://letsencrypt.org/docs/>

▶ Certbot 공식 문서

<https://certbot.eff.org/>

→ Nginx server_name 공식 문서

https://nginx.org/en/docs/http/server_names.html



- EOF -